

A novel time-dependent electric vehicle pickup and delivery problem with time windows in cold supply chains

Sihan Yang^a, Jieman Xia^a, Shanshan Guo^b, Xin Fang^{c,*}, Chengjie Ni^a

^a School of Information Technology and Artificial Intelligence, Zhejiang University of Finance and Economics, Hangzhou 310018, PR China

^b Library, Zhejiang University of Finance and Economics, Hangzhou 310018, PR China

^c School of Business Administration, Zhejiang University of Finance and Economics Dongfang College, Hangzhou 310018, PR China

ARTICLE INFO

Keywords:

Electric vehicle pickup and delivery problem with time windows
Time-dependent travel time
Product spoilage
Deep reinforcement learning with paired embedding and graph attention network

ABSTRACT

In order to alleviate the environmental impact of greenhouse gas, electric vehicles (EVs) have been widely employed in cold supply chains. Many researchers have focused on the pickup and delivery problem with time windows (PDPTW) to design the optimal routing plan satisfying the specified precedence constraints and time windows. However, few studies considered the time-dependent travel time and product spoilage in electric vehicle PDPTW. Therefore, this study proposes a novel time-dependent electric vehicle PDPTW model in cold supply chains. Support vector machine is employed to train the travel speed predictor so that the time-dependent travel time can be obtained through the predicted travel speed. The traditional linear loss function for product spoilage is extended by taking into account the customers' sensitivity to product spoilage. To solve the proposed model, a new deep reinforcement learning with paired embedding and graph attention network (DRL-PE-GAT) algorithm is proposed. The paired embedding and graph attention network (GAT) are integrated to capture the intrinsic relationship between pickup and delivery customer nodes and obtain the spatial dependencies between nodes, respectively. Further, several experiments are conducted with the proposed algorithm and three baseline algorithms to validate the performance of the proposed algorithm.

1. Introduction

Greenhouse gas emissions from supply chains significantly contribute to dealing with climate change. Electric vehicles (EVs) have been widely employed in supply chains owing to their carbon-free features. To meet the realistic needs of supply chains, the pickup and delivery problem with time windows (PDPTW) has been proposed as an extension to the traditional vehicle routing problem with time windows (VRPTW), aiming to satisfy the customers' demands within the specified precedence constraints and time windows.

Traffic flow fluctuates over time due to realistic factors, such as weather and traffic congestion, which leads EV's travel time during transportation to be time-dependent. In order to provide optimal routing plans for supply chains, Sun et al. (2018) proposed the piecewise linear travel time function in the common PDPTW to simulate the time-dependent travel time. However, few studies have dealt with the time-dependent travel time in the electric vehicle pickup and delivery problem with time windows (EV-PDPTW) of cold supply chains, since the

additional factors such as product spoilage and recharging EVs make the problem more complex. Therefore, this study proposes a novel time-dependent EV-PDPTW (TD-EV-PDPTW) model in cold supply chains to design optimal routing plans for EVs. Moreover, to better simulate the realistic traffic situation at different time periods and provide reliable routing plans, this study employs support vector machine (SVM) (Cortes and Vapnik 1995) to train the travel speed predictor using real-world traffic data and obtain the time-dependent travel time of each arc according to the predicted travel speed.

Due to the perishability of products, effective distribution routing plans are significant to cold supply chains. Previous studies focused on optimizing different objective functions, such as customer's satisfaction (Song et al. 2020), and the reliability of suppliers (Hashemi-Amiri et al. 2023). The above studies ignored the product spoilage during transportation, which often leads to a serious loss in the economic benefits of cold supply chains. Deng et al. (2022) proposed a product spoilage loss function in the PDPTW to estimate product spoilage and selling prices in a linear way. However, to our best knowledge, few studies have

* Corresponding author.

E-mail addresses: 240505011025@zufe.edu.cn (S. Yang), xiajieman@zufe.edu.cn (J. Xia), guoshanshan@zufe.edu.cn (S. Guo), 20120505@zufe.edu.cn (X. Fang), 1374706274@zufe.edu.cn (C. Ni).

<https://doi.org/10.1016/j.eswa.2025.128813>

Received 28 December 2024; Received in revised form 23 April 2025; Accepted 26 June 2025

Available online 28 June 2025

0957-4174/© 2025 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

considered customers' sensitivity to product spoilage. Therefore, this study extends the traditional linear loss function (Deng et al. 2022) by considering customers' sensitivity to product spoilage to better simulate the real-world situation.

Researchers have proposed the exact algorithms to solve NP-hard problems, since they can decompose a large problem into parts (Wang et al. 2019), such as the branch-and-cut algorithm (Archetti et al. 2020). Although the exact algorithms can obtain the optimal solutions, the NP-hard problem becomes extremely computationally heavy when the problem scale is large. Thus, heuristic algorithms have been widely employed to solve NP-hard problems, since they can find feasible solution within a short time. For example, in our previous work (Zhang et al. 2024), an extended two-phase adaptive large neighborhood search algorithm was proposed to solve the electric location routing problem. Harbaoui Dridi et al. (2020) handled the PDPTW using the particle swarm optimization algorithm, but the search rules that affect the performance of the algorithm rely heavily on human experiences.

Recently, a few researchers (e.g., Vinyals et al. 2015; Nazari et al. 2018; Masood et al. 2024) tried to employ deep reinforcement learning (DRL) to solve combinatorial problems, and proved its effectiveness. This study proposes a new DRL with paired embedding and graph attention network (DRL-PE-GAT) algorithm to solve the TD-EV-PDPTW model effectively. The paired embedding (Zong et al. 2022) is a method that can merge the information of pickup and delivery customer nodes to better capture the intrinsic relationship. This is achieved by concatenating the information from the paired nodes and embedding them together into a shared feature space. Graph attention network (GAT) (Velićković et al. 2018) is a neural network architecture designed to handle graph-structured data by incorporating an attention mechanism, which can obtain the spatial dependencies and learn the importance between nodes. By incorporating the above two components, the proposed algorithm can provide high-quality solutions for the TD-EV-PDPTW model within a short time.

The remainder of this paper is organized as follows. Section 2 discusses the relevant works. In section 3, the TD-EV-PDPTW model is proposed. Section 4 illustrates the DRL-PE-GAT algorithm in detail. Section 5 shows the experimental results. The conclusion of this paper and the promising directions of future research are presented in section 6.

2. Related work

2.1. Pickup and delivery problem with time windows

As an extension to the traditional VRPTW, the PDPTW aims to design an optimal routing plan which satisfies the pickup and delivery customers' demands, time window constraints, and precedence constraints (Dumas et al. 1991; Wang et al. 2021). To design optimal routing plans for supply chains, researchers have proposed various methods to address time-dependent travel time. Gmira et al. (2021) proposed a travel time function with three intervals, where each arc's travel time is a step function of the departure time. Liu et al. (2021) split the traffic time into several periods and designed a partial service policy to simulate the realistic situation. Wang et al. (2022) employed the queuing theory to calculate the time-dependent delay and travel time during the congestion periods in the truck-drone hybrid routing problem. The above studies simply assumed that travel time is a piecewise linear function, which reduce the reliability of routing plans in real-world transportation. Figliozzi (2011) used the travel speed data from Portland's metropolitan area to explore the time-dependent distribution networks, but assumed different fixed travel speeds for different scenarios. Qian and Eglese (2016) depicted the all-day traffic condition using the traffic data in the London charge area and obtained the finer travel time function, but the delivery was only performed during peak hours.

As an important part of supply chains, cold supply chains deliver temperature-sensitive products such as foods and vaccines, which

requires more effective distribution routing plans. Therefore, several studies have considered the PDPTW in cold supply chains. Song et al. (2020) considered the simultaneous pickup and delivery VRPTW in the cold supply chains, with the aim to minimize the fixed cost and energy consumption. Deng et al. (2022) considered the PDPTW with incompatibility constraints in cold transportation aiming to minimize the travel cost, vehicle's refrigeration cost, and product spoilage loss. Zheng et al. (2022) proposed the PDPTW considering uncertain food preparation time. However, to our best knowledge, few studies have dealt with the time-dependent travel time in the EV-PDPTW of cold supply chains, since the product spoilage and recharging EVs will lead the problem more complex.

2.2. Product spoilage in cold supply chains

For the temperature-sensitive products in cold supply chains, they may spoil during the transportation, which can result in significant losses to cold supply chains. Therefore, several researchers have focused on product spoilage to improve the efficiency of cold supply chains.

Rahbari et al. (2019) considered the product spoilage in the vehicle routing and scheduling problem, where the penalty costs were defined to estimate the loss of product spoilage. Leng et al. (2020) proposed the location-routing problem in cold supply chains, aiming to minimize the logistics cost and the amount of product spoilage. To quantify the loss of product spoilage, Osvald and Stirn (2008) assumed that product has limited lifespan and spoils linearly over time. Stellingwerf et al. (2021) proposed a time-and-temperature dependent kinetic model to quantify the product spoilage in transportation. Deng et al. (2022) proposed a linear loss function to quantify the relationship between product spoilage and selling price. Wang et al. (2023) measured perishable products' spoilage through the time windows and assumed that customers' satisfaction is negatively related to products' spoilage. However, few studies have considered customers' sensitivity to product spoilage in cold supply chains, which needs to be considered in the real-world situation.

2.3. Algorithms for solving the pickup and delivery problem with time windows

To obtain the optimal routing plans of the PDPTW, some studies have employed exact algorithms to solve the problem. Baldacci et al. (2011) proposed an exact algorithm based on set-partitioning-like integer formulation to find the near-optimal dual solution for the PDPTW. Veenstra et al. (2017) proposed a branch-price-and-cut algorithm to solve the PDPTW.

However, the PDPTW is a typical NP-hard problem, and cannot be solved effectively when problem scale becomes large. Therefore, many heuristic algorithms have been proposed to tackle the problem. Shi et al. (2018) proposed a simulated annealing-based metaheuristic algorithm to solve PDPTW with stochastic travel and service times. W.L. Li et al. (2021) proposed a memetic algorithm with global search strategies and local search heuristics in the VRPTW with simultaneous pickup and delivery, aiming to minimize the total cost during transportation. However, the design of search rules and the parameter tuning in heuristic algorithms rely heavily on human experience, which may lead to unstable performance of algorithms.

DRL is a promising approach to address the above limitations by automatically learning and optimizing the decision-making policy during the training process. J.W. Li et al. (2021) proposed the DRL with heterogeneous attentions to solve the pickup and delivery problem. Lin et al. (2021) proposed the DRL including the graph embedding, the attention mechanism, and the long short-term memory decoder to obtain the feasible solutions effectively. Delamer et al. (2024) proposed a GNN-based approach to address dynamic emergency vehicle routing problems, demonstrating the effectiveness of GNNs in capturing real-time traffic dynamics and optimizing routes under uncertainty. Luo

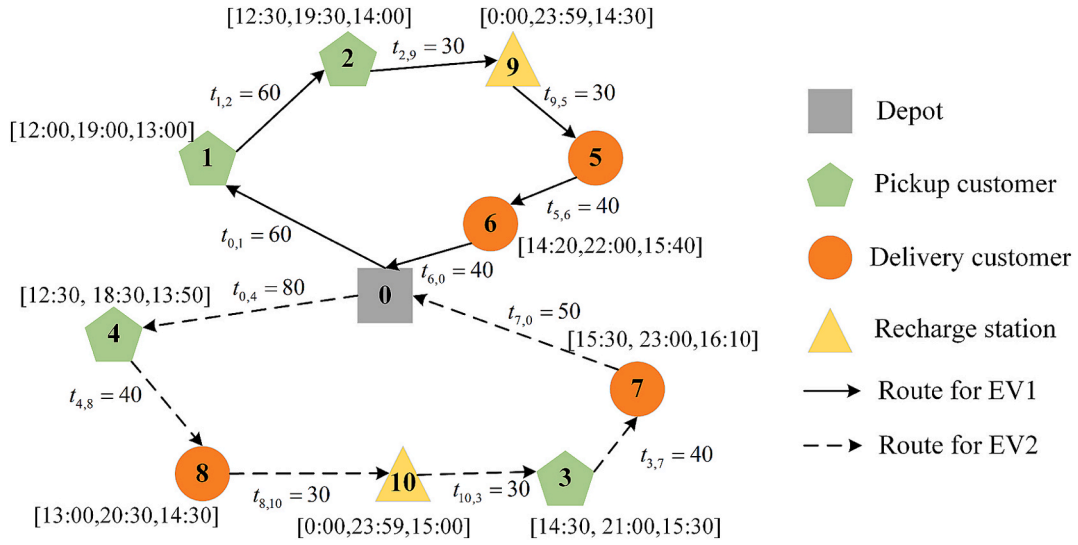


Fig. 1. EV's optimal routing plan at 12:00.

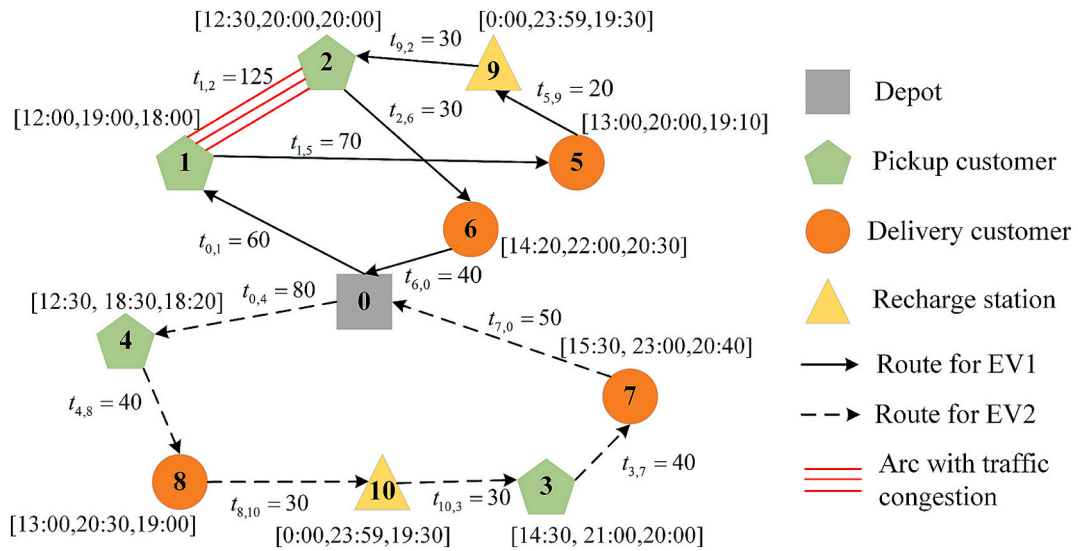


Fig. 2. EV's optimal routing plan at 17:00.

and Li (2025) developed an efficient encoder-decoder network for the capacitated vehicle routing problem, leveraging GNNs to enhance solution quality and computational efficiency. However, these studies did not consider merging the information of pickup and delivery customer nodes and obtaining the spatial dependencies between nodes. Inspired by the above work and considering the characteristic of the TD-EV-PDPTW model, this study proposes the DRL-PE-GAT algorithm which combines the DRL with the paired embedding and GAT.

3. Time-dependent electric vehicle pickup and delivery problem with time windows

3.1. Problem description

The TD-EV-PDPTW model is defined on the directed graph $G(N, A)$, where N represents the set of all nodes, including set of depot $D = \{0\}$, set of pickup customer nodes $N_p = \{1, 2, \dots, n\}$, set of delivery customer nodes $N_d = \{n+1, n+2, \dots, 2n\}$, and set of recharge stations $N_r = \{2n+1, 2n+2, \dots, 2n+r\}$. A represents the set of arcs, where $A = \{(i, j) | i, j \in N, i \neq j\}$. During the transportation, EV leaves depot 0, serves

ith pickup customer node and delivers the products to its paired delivery customer node $n+i$ within the specified time windows, travels to the recharge stations when necessary, and finally returns to the depot.

The traffic conditions vary over time, which leads to significant changes in the travel time in each arc and EV's optimal routing plan. Figs. 1 and 2 give the comparison examples of optimal routing plans at different time periods in a day. In Fig. 1, the numbers in the bracket represent the customers' earliest, latest time windows, and EV's arrival time respectively, the numbers along the arcs are the travel times between two nodes. At 12:00, no traffic congestion occurs, and to reduce traveling costs, EV1 can visit all pickup customer nodes first and then visit all delivery customer nodes. In Fig. 2, traffic congestion happens between pickup customer nodes 1 and 2 at 17:00. In order to visit pickup and delivery customer nodes within the specified time window as well as to prevent product spoilage, EV1 visits its paired delivery customer node 5 immediately after visiting pickup customer node 1.

3.2. Product spoilage loss function

In cold supply chains, product spoilage can lead to a significant loss.

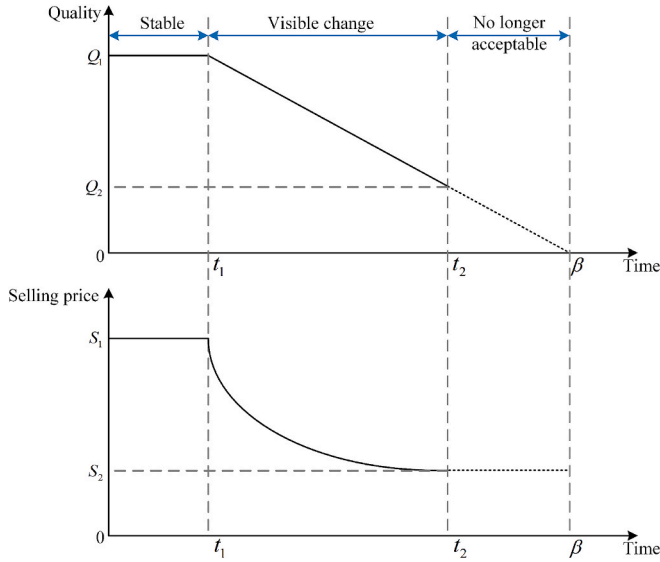


Fig. 3. An illustrative example of product spoilage loss.

To provide a precise evaluation of product spoilage loss, this study extends the traditional linear loss function by considering customers' sensitivity to product spoilage, and an illustrative example is shown in Fig. 3. In Fig. 3, the moment that EV leaves pickup customer node is denoted as 0, and product's lifespan β is divided into three stages, i.e., stage 1 ($0-t_1$), stage 2 (t_1-t_2), and stage 3 ($t_2-\beta$).

In stage 1, product's quality is stabilized at Q_1 and the selling price remains at S_1 . In stage 2, product spoilage happens and the visible change occurs, leading to the selling price decreasing convexly, and the product has the minimum selling price S_2 at t_2 . As for delivery customer nodes, they have different sensitivity parameters to different kinds of products' spoilage, resulting in different curvatures of the loss function. In stage 3, product's quality falls below Q_2 and severe spoilage makes the product no longer acceptable and saleable.

The spoilage loss $Loss_i$ of i th pickup customer node's product can be calculated according to the product spoilage loss function, as shown in Eq. (1).

$$Loss_i = \begin{cases} 0, & 0 \leq at^{n+i} \leq t_1^i \\ r_i \left[S_1^i - \left(\frac{at^{n+i} - t_1^i}{t_2^i - t_1^i} \right)^\xi (S_1^i - S_2^i) \right], & t_1^i < at^{n+i} \leq t_2^i \\ r_i S_2^i, & t_2^i < at^{n+i} \leq \beta^i \end{cases} \quad (1)$$

where t_1^i and t_2^i represent two time points for quality separation of the product at i th pickup customer node, β^i is the lifespan of the product at i th pickup customer node, at^{n+i} represents EV's arrival time at the paired delivery customer node $n+i$. S_1^i and S_2^i represent the maximum and minimum selling price of the product at i th pickup customer node respectively. ξ represents customer's sensitivity parameter to product spoilage. r_i is the demand of the i th pickup customer node.

3.3. Mathematical model

3.3.1. Notation

To describe the TD-EV-PDPTW model more explicitly, the notations are given as follows:

(1) Sets

N	Set of all nodes
N_r	Set of recharge stations
N_p	Set of pickup customer nodes
N_d	Set of delivery customer nodes

(continued on next column)

(continued)

D	Set of the depot
(2) Parameters	
B	EV's maximum battery capacity
α	EV's energy consumption rate (per unit distance)
γ	EV's traveling cost (per unit distance)
r_i	Demand of node i
es_i	Earliest time window of node i
ls_i	Latest time window of node i
d_{ij}	Distance between nodes i and j
(3) Decision variables	
dt_i	The EV's departure time at node i
l_i	The EV's load state at node i
b_i	The EV's remaining battery state at node i
$\tau_{ij}(dt_i)$	The EV's travel time from nodes i to j when its departure time at node i is dt_i
x_{ij}	Binary variable, which equals to 1 if arc (i,j) is traveled by the EV; and 0 otherwise
$Loss_i$	Spoilage loss of pickup customer node i

3.3.2. Model formulation

The objective function of the TD-EV-PDPTW model consists of two parts. As Eq. (2) shows, the first component is to minimize EV's traveling cost during the transportation, and the second component is to minimize the loss of product spoilage.

$$\min f = \gamma \sum_{i \in N} \sum_{j \in N, i \neq j} x_{ij} d_{ij} + \sum_{i \in N_p} r_i Loss_i \quad (2)$$

The objective function is subjected to the following constraints:

$$\sum_{i \in (N_p \cup N_d)} x_{ij} = 1, \forall j \in N, i \neq j \quad (3)$$

$$\sum_{j \in (N_p \cup N_r)} x_{ij} = 1, i = 0 \quad (4)$$

$$\sum_{i \in (N_d \cup N_r)} x_{ij} = 1, j = 0 \quad (5)$$

$$\sum_{i \in (N \setminus D)} x_{ik} - \sum_{j \in (N \setminus D)} x_{kj} = 0, \forall k \in (N \setminus D) \quad (6)$$

$$\sum_{j \in (N \setminus D)} x_{ij} - \sum_{j \in (N \setminus D)} x_{n+i,j} = 0, \forall i \in N_p \quad (7)$$

$$0 \leq l_i \leq L, \forall i \in N \quad (8)$$

$$\text{If } x_{ij} = 1, \text{ then } 0 \leq l_j \leq l_i - r_i x_{ij}, \forall i, j \in N, i \neq j \quad (9)$$

$$\text{If } x_{ij} = 1, \text{ then } 0 \leq b_j \leq b_i - \alpha d_{ij}, \forall i \in (N \setminus N_r), \forall j \in N, i \neq j \quad (10)$$

$$\text{If } x_{ij} = 1, \text{ then } 0 \leq b_j \leq B - \alpha d_{ij}, \forall i \in N_r, \forall j \in N, i \neq j \quad (11)$$

$$es_i \leq dt_i \leq ls_i, \forall i \in N \quad (12)$$

$$\text{If } x_{ij} = 1, \text{ then } dt_i + \tau_{ij}(dt_i) \leq dt_j, \forall i, j \in N, i \neq j \quad (13)$$

$$dt_{n+i} \geq dt_i, \forall i \in N_p \quad (14)$$

$$x_{ij} \in \{0, 1\}, \forall i, j \in N \quad (15)$$

Constraint (3) ensures that each node is served only once by EVs. Constraints (4)-(6) guarantee the flow conservation at the origin, destination and intermediate nodes respectively. Constraint (7) ensures that same pair of pickup and delivery customer nodes are served by same EV. Constraints (8)-(9) ensure that EV's load state is not violated. Constraints (10)-(11) check EV's remaining battery state and ensure it is not violated. Constraint (12) guarantees that each node is served within the given time windows. Constraint (13) ensures the time feasibility of each arc. Constraint (14) represents that pickup customer nodes are served before delivery customer nodes. Constraint (15) defines the decision

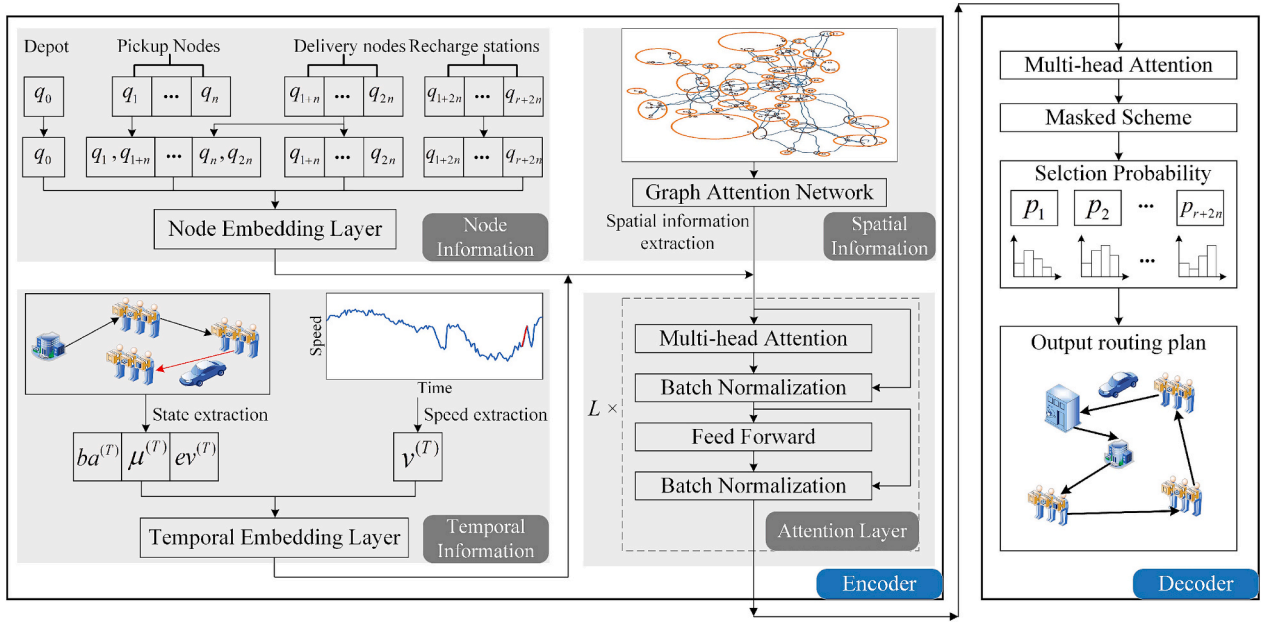


Fig. 4. Structure of the DRL-PE-GAT algorithm.

variables.

4. Solution methodology

This section firstly reformulates the TD-EV-PDPTW model as the form of Markov Decision Process (MDP) and then proposes the DRL-PE-GAT algorithm.

4.1. Reformulation of the TD-EV-PDPTW model

Constructing the routing plan of the vehicle routing problem can be regarded as a sequential decision process and reformulated as a randomized greedy algorithm, which is referred to as MDP (Guo et al. 2023) in this study. The MDP includes four components, i.e., the state space, the action space, the transition rules and the reward function. Among them, the MDP selects actions based on the current state instead of completely relying on deterministic rules. In addition, the transition rules in MDP ensure the rationality of state updates, while the reward function evaluates the quality of actions.

State space: In the TD-EV-PDPTW model, the state space $sp^{(k)}$ represents the routing environment at step k , which includes the node information Q and the step k 's temporal information $T^{(k)}$, i.e., $sp^{(k)} = (Q, T^{(k)})$. Despite varying instance sizes, all node information (number of nodes of the network, number of demands, etc.) and temporal information are mapped into fixed-dimensional embeddings through the paired embedding and GAT. The specific parameters are described in Sub-section 5.1. These embeddings ensure that the Markov chain parameters remain fixed during training, enabling the model to handle variable-sized instances while maintaining stable learning. The node set $Q = \{q_0, q_1, \dots, q_{2n+r}\}$, where $q_i = (c_i, r_i, es_i, ls_i)$ represents the i th node information, and the coordinate c_i represents the position of the i th node in a 2-dimensional space. The temporal information $T^{(k)} = (ba^{(k)}, \mu^{(k)}, ev^{(k)}, v^{(k)})$, where $ba^{(k)}$ represents the remaining battery of EV used at step k , $\mu^{(k)}$ represents the system time at step k . System time is used to simulate and record the time during transportation. $ev^{(k)}$ and $v^{(k)}$ denote the available number of EVs and EV's predicted travel speed at step k , respectively.

Action space: The action $y^{(k)}$ is used to select the node to be visited at step k and $\pi^{(k)} = \{y^{(1)}, y^{(2)}, \dots, y^{(k-1)}, y^{(k)}\}$ represents the routing plan

formed up to step k .

Transition rules: After the action $y^{(k-1)}$ is executed, step k 's temporal information $T^{(k)}$ in $sp^{(k)}$ will change accordingly. The transition rules are used to guide the update of $T^{(k)}$, as shown in Eqs. (16)-(18).

Firstly, $ba^{(k)}$ is updated according to Eq. (16), where $d_{y^{(k-1)}, y^{(k)}}$ denotes the distance between nodes selected by $y^{(k-1)}$ and $y^{(k)}$.

$$ba^{(k)} = \begin{cases} ba^{(k-1)} - ad_{y^{(k-1)}, y^{(k)}}, & y^{(k)} \in \{N_p \cup N_d\} \\ B - ad_{y^{(k-1)}, y^{(k)}}, & \text{otherwise} \end{cases} \quad (16)$$

Next, $\mu^{(k)}$ is updated according to Eq. (17), where cr and $es_{y^{(k)}}$ represent EV's recharging rate and the earliest time windows of the node selected by $y^{(k)}$ respectively. $dt_{y^{(k-1)}}$ denotes the EV's departure time at the node selected by $y^{(k-1)}$. $\tau_{y^{(k-1)}, y^{(k)}}(dt_{y^{(k-1)}})$ denotes the travel time between nodes selected by $y^{(k-1)}$ and $y^{(k)}$ when the departure time is $dt_{y^{(k-1)}}$.

$$\mu^{(k)} = \begin{cases} \max(\mu^{(k-1)}, es_{y^{(k)}}) + \tau_{y^{(k-1)}, y^{(k)}}(dt_{y^{(k-1)}}), & y^{(k)} \in \{N_p \cup N_d\} \\ \mu^{(k-1)} + \frac{B - ba^{(k-1)}}{cr} + \tau_{y^{(k-1)}, y^{(k)}}(dt_{y^{(k-1)}}), & y^{(k)} \in N_r \\ \tau_{y^{(k-1)}, y^{(k)}}(dt_{y^{(k-1)}}), & y^{(k)} \in D \end{cases} \quad (17)$$

Then $ev^{(k)}$ is updated according to Eq. (18).

$$ev^{(k)} = \begin{cases} ev^{(k-1)} - 1, & y^{(k)} \in D \\ ev^{(k-1)}, & \text{otherwise} \end{cases} \quad (18)$$

To better simulate the realistic traffic situation and provide reliable routing plans, a travel speed predictor (Guo et al. 2023) is used. SVM is employed to train the travel speed predictor using real-world traffic data, and predict the travel speed of EVs. Finally, $v^{(k)}$ is updated according to the travel speed predictor and system time $\mu^{(k)}$ at step k , as shown in Eq. (19).

$$v^{(k)} = SVM(\mu^{(k)}) \quad (19)$$

where $SVM()$ represents the travel speed predictor trained by SVM. The travel speed predictor will update $v^{(k)}$ according to system time $\mu^{(k)}$.

Reward function: A smaller negative value of reward function implies a higher-quality routing plan. In this study, $Rf(\pi^{(k)})$ represents the reward function of the formed routing plan $\pi^{(k)}$, which contains four components, as shown in Eq. (20).

$$Rf(\pi^{(k)}) = - \left(\sum_{\kappa=1}^k d_{y^{(\kappa-1)}, y^{(\kappa)}} + \sum_{\kappa=1}^k \sum_{y^{(\kappa)} \in N_d} P_{y^{(\kappa)}} + \theta_1 \max\{0, -ba^{(k)}\} + \theta_2 \max\{0, -ev^{(k)}\} \right) \quad (20)$$

The first component is the total traveling distance of $\pi^{(k)}$, and the second component represents the total spoilage loss of $\pi^{(k)}$. The third and fourth components are the penalties for violating the constraints of remaining battery state and available numbers of EVs respectively. Among them, the penalty term can address implied risks, such as battery depletion due to traffic delays, thus enhancing robustness in dynamic environments. θ_1 and θ_2 are corresponding penalty parameters, which are set to 1000 and 30 in this study respectively.

4.2. Deep reinforcement learning with paired embedding and graph attention network

The structure of the DRL-PE-GAT algorithm, which mainly includes encoder and decoder (Guo et al. 2023), is shown in Fig. 4. In Fig. 4, the node, temporal, and spatial information are collected and fed into the attention layer in the encoder.

As for node information, different kinds of nodes are fed into the node embedding layer. Especially, the pickup and delivery customer nodes are merged to represent the intrinsic relationship due to their precedence constraints. As for the temporal information, the time-related information is chosen as the input of the temporal embedding layer. As for the spatial information, the GAT is used to obtain the spatial dependencies and learn the importance between nodes. The attention layer is used to integrate the information and emphasize the characteristics of important nodes and edges, thus enhancing the ability to focus on and process the key information. The decoder will process the information from the encoder through the attention layer and calculate the probability of visiting each node.

4.2.1. The encoder

The encoder consists of four components, i.e., node information, temporal information, spatial information, and attention layer, as shown below.

1) *Node information*: In the TD-EV-PDPTW model, the pickup and delivery customer nodes are intrinsically related due to their precedence constraints. To better capture the relationship, this study employs the paired embedding (Zong et al. 2022) in the node embedding layer as shown in Eq. (21). Specifically, paired embedding involves combining the features of pickup and corresponding delivery nodes into a unified representation in the node embedding layer.

$$h_i = \begin{cases} W_{Node}^0 q_i + b_{Node}^0, & i \in D \\ W_{Node}^p [q_i; q_{n+i}] + b_{Node}^p, & i \in N_p \\ W_{Node}^d q_i + b_{Node}^d, & i \in N_d \\ W_{Node}^r q_i + b_{Node}^r, & i \in N_r \end{cases} \quad (21)$$

where h_i represents the embedding of the i th node. W_{Node}^0 , W_{Node}^p , W_{Node}^d , and W_{Node}^r are the learnable matrices in the node embedding layer, corresponding to the depot, pickup customer nodes, delivery customer nodes, and recharge stations, respectively. The learnable matrices are defined to transform the input data into a high-dimensional space, and by processing through attention layers, the encoder can gradually extract the features of input data. b_{Node}^0 , b_{Node}^p , b_{Node}^d , and b_{Node}^r represent the biases for depot, pickup customer nodes, delivery customer nodes and recharge stations in the node embedding layer, respectively. Additionally, paired embedding is performed by concatenating the information of pickup and delivery customer nodes. The vector q_i of the pickup node is concatenated with the corresponding vector q_{n+i} of the

delivery node, i.e., $h_i = W_{Node}^p [q_i; q_{n+i}] + b_{Node}^p$. Where “;” denotes the concatenating operation of two vectors along the same dimension, and $W_{Node}^p [q_i; q_{n+i}]$ represents the learnable matrices of node information in the node embedding layer for concatenating pickup nodes and delivery customer nodes.

2) *Temporal information*: To better extract the temporal information, this study proposes a temporal embedding layer to incorporate EV's battery state, system time, available number of EVs, and EV's predicted travel speed into a continuous vector, as shown in Eq. (22).

$$\mathfrak{N}^{(k)} = W_{Temp}^{(k)} [ba^{(k)}; \mu^{(k)}; ev^{(k)}; v^{(k)}] + b_{Temp}^{(k)} \quad (22)$$

where $\mathfrak{N}^{(k)}$ represents the temporal embedding at step k , $W_{Temp}^{(k)}$ and $b_{Temp}^{(k)}$ denote the learnable matrices and bias at step k in the temporal embedding layer, respectively.

3) *Spatial information*: The GAT (Lei et al. 2022) is employed to obtain the spatial dependencies and learn the importance between nodes. The GAT contains Z layers (Lei et al. 2022), in which the nodes' 2-dimensional coordinates are chosen as the input node feature, the distances between nodes are selected as the input edge feature, and two feature embeddings $\hat{c}_i^{(0)}$ and $\hat{d}_{ij}^{(0)}$ are used to map two kinds of features into high-dimensional spaces, as shown in Eqs. (23)-(24).

$$\hat{c}_i^{(0)} = W_{\tau} c_i + b_{\tau}, \forall i \in N \quad (23)$$

$$\hat{d}_{ij}^{(0)} = W_{Edge} d_{ij} + b_{Edge}, \forall i, j \in N, i \neq j \quad (24)$$

where $\hat{c}_i^{(0)}$ denotes the i th node's initial node feature embedding in the GAT, W_{τ} and b_{τ} represent the learnable matrices and bias for the initial node feature embedding. $\hat{d}_{ij}^{(0)}$ is the initial edge feature embedding between nodes i and j , and stay unchanged in each layer of GAT. W_{Edge} and b_{Edge} denote the learnable matrices and bias for the initial edge feature embedding respectively.

$\alpha_{ij}^{(z+1)}$ represents the attention coefficient between nodes i and j at the $(z+1)$ th GAT layer, which can be calculated according to Eq. (25).

$$\alpha_{ij}^{(z+1)} = \frac{\exp(\text{LeakyRelu}(W_G^{(z)} [\hat{c}_i^{(z)}; \hat{c}_j^{(z)}; \hat{d}_{ij}^{(0)}]))}{\sum_{k \in N \setminus \{i\}} \exp(\text{LeakyRelu}(W_G^{(z)} [\hat{c}_i^{(z)}; \hat{c}_k^{(z)}; \hat{d}_{ik}^{(0)}]))} \quad (25)$$

where $\text{LeakyRelu}()$ represents the LeakyRelu activation function, $W_G^{(z)}$ denotes the learnable matrices for the z th GAT layer. $\hat{c}_i^{(z)}$ represents the node feature embedding for i th node in the z th GAT layer, which can be obtained as Eq. (26) shows.

$$\hat{c}_i^{(z)} = \sum_{j \in N \setminus \{i\}} \alpha_{ij}^{(z)} W_G^{(z)} \hat{c}_j^{(z-1)} \quad (26)$$

After Z layers of transformation, the final output feature ψ of the GAT encapsulates the features of each node, i.e., $\psi = \{\hat{c}_0^{(Z)}, \hat{c}_1^{(Z)}, \dots, \hat{c}_{2n+r}^{(Z)}\}$.

4) *Attention layers*: The DRL-PE-GAT algorithm contains L attention layers (Lei et al. 2022), and each attention layer includes the multi-head attention (MHA) sub-layer, batch normalization sub-layer, and the feed-forward sub-layer. To obtain the relationship between different kinds of information effectively, let $M_{i,k}^{(0)}$ be the initial multimodal embedding of the i th node, which concatenates the output h_i of the node embedding layer, the output $\mathfrak{N}^{(k)}$ of the temporal embedding layer and the output ψ of the GAT.

The MHA sub-layer contains H attention heads, each of which can be defined as Eq. (27) shows.

$$Q_{i,k}^{(h)} = W_Q^{(h)} M_{i,k}^{(0)}; K_{i,k}^{(h)} = W_K^{(h)} M_{i,k}^{(0)}; V_{i,k}^{(h)} = W_V^{(h)} M_{i,k}^{(0)} \quad (27)$$

where $Q_{ik}^{(h)}$, $K_{ik}^{(h)}$, and $V_{ik}^{(h)}$ represent the i th node's query, key, and value vectors of the h th attention head at step k , respectively. $W_Q^{(h)}$, $W_K^{(h)}$, and $W_V^{(h)}$ denote the learnable matrices for the query, key, and value vectors of the h th attention head, respectively.

$a_{ik}^{(h)}$ represents the i th node's attention score of the h th attention head at step k , and can be calculated according to Eq. (28).

$$a_{ik}^{(h)} = \text{Softmax} \left(\frac{Q_{ik}^{(h)} (K_{jk}^{(h)})^{\text{Trans}}}{\sqrt{d_K}} \right) V_{jk}^{(h)} \quad (28)$$

where $\text{Softmax}()$ represents the softmax activation function, "Trans" denotes the transpose operation, and d_K is the dimension of the key vectors.

The final output of the MHA sub-layer concatenates the attention score of each attention head, which is shown in Eq. (29).

$$A_{ik}^{(l)} = W_O^{(l)} [a_{ik}^{(1)}; a_{ik}^{(2)}; \dots; a_{ik}^{(H)}] \quad (29)$$

where $A_{ik}^{(l)}$ and $W_O^{(l)}$ represent the i th node's output vector of the l th attention layer at step k and the learnable matrices for the output vectors at the l th attention layer, respectively.

In the batch normalization and feed forward sub-layers, the i th node's multimodal embedding $M_{ik}^{(l)}$ of the l th attention layer at step k is updated according to Eq. (30).

$$M_{ik}^{(l)} = BN^{(l)} [BN^{(l)} (M_{ik}^{(l-1)} + A_{ik}^{(l)}) + FF^{(l)} (BN^{(l)} (M_{ik}^{(l-1)} + A_{ik}^{(l)}))] \quad (30)$$

where $BN^{(l)}$ and $FF^{(l)}$ represent the batch normalization and feed forward operations at the l th attention layer, respectively.

After L attention layers of transformation, the average graph embedding $\bar{M}^{(k)}$ which aggregates each node's multimodal embedding at step k is generated as Eq. (31) shows.

$$\bar{M}^{(k)} = \frac{1}{2n+r+1} \sum_{i=0}^{2n+r} M_{ik}^{(L)} \quad (31)$$

4.2.2. The decoder

In the decoder, the routing plan is constructed using the information provided by the encoder. At step k , the multimodal embedding $M_{y^{(k-1)}}^{(L)}$ of the node selected by $y^{(k-1)}$, and average graph embedding $\bar{M}^{(k)}$ are concatenated as the input of the decoder, as Eq. (32) shows.

$$\hat{M}^{(k)} = W_{\text{Decoder}} [M_{y^{(k-1)}}^{(L)}; \bar{M}^{(k)}] \quad (32)$$

where $\hat{M}^{(k)}$ and W_{Decoder} denote the input of the decoder at step k , and the learnable matrices for concatenating the multimodal and average graph embeddings in the decoder, respectively.

Then, $\hat{M}^{(k)}$ and $M_{y^{(k-1)}}^{(L)}$ will pass through the same MHA sub-layer as in the encoder, which is shown in Eq. (33).

$$\tilde{M}^{(k)} = \text{Multihead} \left(\tilde{W}_Q \hat{M}^{(k)}, \tilde{W}_K M_{y^{(k-1)}}^{(L)}, \tilde{W}_V M_{y^{(k-1)}}^{(L)} \right) \quad (33)$$

where $\tilde{M}^{(k)}$ is the output of the MHA sub-layer at step k , $\text{Multihead}()$ represents the operation of the MHA sub-layer. $\tilde{W}_Q$, $\tilde{W}_K$, $\tilde{W}_V$ are learnable matrices for the query, key, and value vectors in the decoder, respectively.

$\zeta_i^{(k)}$ is the hidden state of i th node at step k , which is defined as Eq. (34) shows.

Table 1

Parameters setting of four algorithms.

Algorithms	Parameters	Value
DRL-PE-GAT	Batch size	128
	Embedding size	128
	Learning rate	0.001
	Number of GAT layers	3
	Number of attention layers	3
	Number of attention heads	8
AM	Batch size	128
	Embedding size	128
	Learning rate	0.001
	Number of attention layers	3
	Number of attention heads	8
ALNS	Minimum and maximum removal percentage of random destroy operator	(0.1;0.3)
	Minimum and maximum removal percentage of worst destroy operator	(0.2;0.5)
	Bonus of obtaining the best solution, worse solution and worse but accepted solution	(20;15;5)
VNS	Initial temperature	500
	Cooling rate	0.9

$$\zeta_i^{(k)} = C * \tanh \left(\frac{(Q^{(k)})^{\text{Trans}} K_{ik}}{\sqrt{d_K}} \right) \quad (34)$$

where C is the clip rate to stabilize the learning process and is set to 10 in this study, $\tanh()$ is the tanh activation function. The query vector $Q^{(k)}$ at step k is obtained through $Q^{(k)} = \tilde{W}_Q \tilde{M}^{(k)}$, and the i th node's key vector K_{ik} at step k is obtained through $K_{ik} = \tilde{W}_K M_{ik}^{(L)}$.

Finally, the decoder calculates the probability of choosing each node according to Eq. (35), which uses the policy gradient method (Kool et al. 2019).

$$p(y^{(k)} = i | sp^{(k-1)}, \pi^{(k-1)}) = \text{Softmax} \left(\text{mask}^{(k)}(\zeta_i^{(k)}) \right) \quad (35)$$

where $p(y^{(k)} = i | sp^{(k-1)}, \pi^{(k-1)})$ is the conditional probability of executing action $y^{(k)} = i$ at step k given that the state space and formed routing plan at step $(k-1)$, and "|" is conditional separator. $\text{mask}^{(k)}()$ is the masked scheme that excludes the infeasible nodes at step k .

The steps are repeated until all pickup and delivery customer nodes are visited by EVs, and finally EVs return to the depot. To improve the training efficiency and stability of the DRL-PE-GAT algorithm, this study employs the well-known rollout algorithm (Kool et al. 2019) to train the proposed algorithm.

5. Experiments

In this section, the proposed algorithm is compared with three baseline algorithms, i.e., the attention model (AM) (Kool et al. 2019), the adaptive large neighborhood search (ALNS) (Ropke and Pisinger 2006), and the variable neighborhood search (VNS) (Mladenović and Hansen 1997). Moreover, sensitivity analyses are performed to analyze the TD-EV-PDPTW model. A personal computer with Windows 11 64-bit and Intel (R) Core i5-12500H CPU, NVIDIA GeForce RTX 3090 GPU is employed as the experimental platform. The proposed algorithm is implemented in Pytorch.

While ALNS and VNS are classical heuristic algorithms primarily designed for generic vehicle routing problems, they have been widely recognized for their robustness and adaptability in combinatorial optimization. To ensure a fair comparison, these algorithms are extended to accommodate the unique constraints of the TD-EV-PDPTW model. Specifically, time-dependent travel time and the product spoilage loss function were integrated to facilitate objective evaluations and constraint checks. This adaptation allows us to evaluate how effectively these heuristics handle dynamic traffic conditions and perishable

products.

5.1. Experimental setting

The parameters of four algorithms are determined after trial-run preliminary experiments. The parameter setting of four algorithms are shown in Table 1.

In addition, the following convergence criteria are used for ALNS and VNS. ALNS terminates after 1000 iterations or when no improvement to the best solution is observed for 200 continuous iterations. VNS terminates when the temperature drops below 1 or when no improvement is observed for 500 iterations.

This study uses 100 epochs to train the proposed algorithm and AM, and each epoch contains 500 batches. In each batch, 128 training benchmark instances are randomly generated. Considering the different problem scales in the real-world situation, two different size of training benchmark instances are used to train two algorithms, i.e., the instances containing 50 nodes (including 1 depot, 7 recharge stations, and 42 pickup and delivery customer nodes) and the instances containing 100 nodes (including 1 depot, 15 recharge stations, and 84 pickup and delivery customer nodes). The trained algorithms are denoted according to the size of training benchmark instances, i.e., “DRL-PE-GAT (50)”, “DRL-PE-GAT (100)”, “AM (50)”, and “AM (100)”.

For each training benchmark instance, the nodes' 2-dimensional coordinates are randomly generated within the region of $[0,1] \times [0,1]$, and the demands of pickup and delivery customer nodes are randomly generated within $[1,10]$. The time window is generated within $[0,1]$, the center of the time window is uniformly generated within $[0,1]$, and the length of the time window is normally distributed with mean 0.2 and deviation 0.1.

To validate the proposed algorithm, 24 testing benchmark instances are randomly generated, and each instance is named according to the numbers of nodes, recharge stations, and EVs (e.g., N10-R3-E3 indicates that the instance contains 10 nodes, 3 recharge stations, and 3 EVs). The testing benchmark instances generated in this study are publicly available in the Figshare database (<https://doi.org/10.6084/m9.figshare.25242805.v1>).

5.2. Performance comparison of the DRL-PE-GAT algorithm

To provide precise prediction for the time-dependent travel time, this study collects data through the Zenodo website, and adopts the Urban Traffic Speed Dataset of Guangzhou, China (<https://zenodo.org/records/1205229>), including more than 100 thousand travel speed data within 2 months to train the travel speed predictor. The travel speed dataset is publicly available in the Figshare database (<https://doi.org/10.6084/m9.figshare.25242832.v1>).

Firstly, this study trains the travel speed predictor using SVM to obtain the time-dependent travel time through the predicted travel speed, in which the Gaussian kernel is used, and the penalty parameter and tolerance parameter are set to 5 and 0.01, respectively. Two indicators are used to evaluate the performance of the predictor, including root mean square error (RMSE), and mean absolute error (MAE), as shown in Eqs. (36)–(37).

$$RMSE = \sqrt{\frac{1}{d_{Num}} \sum_{i=1}^{d_{Num}} (P_{S_i} - T_{S_i})^2} \quad (36)$$

$$MAE = \frac{1}{d_{Num}} \sum_{i=1}^{d_{Num}} |P_{S_i} - T_{S_i}| \quad (37)$$

where d_{Num} is the number of the input data, P_{S_i} and T_{S_i} represent the predicted travel speed and the true travel speed for the i th input data, respectively.

The resulted values of RMSE and MAE are 4.18 and 2.57,

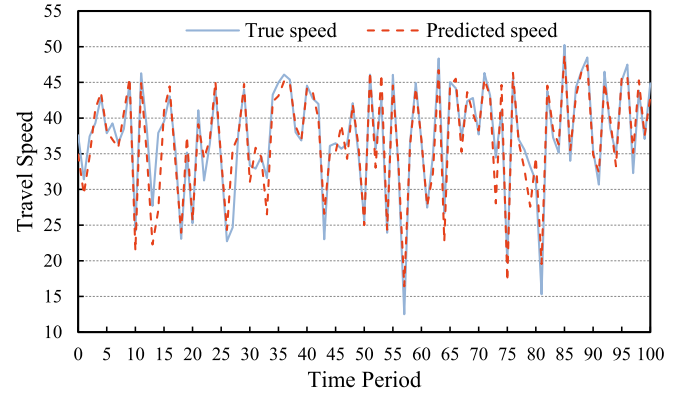


Fig. 5. True and predicted speeds in different time periods.

respectively, which are very small, indicating that there is little error between the true travel speed and predicted travel speed. To further validate its practical applicability, Fig. 5 shows the comparison result of the true travel speed and predicted travel speed over a period of time. Although there is deviation between true speed and predicted speed during some congestion time periods (i.e., when travel speed decreases sharply), the predictor is able to provide reliable prediction in most time periods. This demonstrates the predictor's robustness in real-world scenarios.

Then, the training curves of distance and spoilage loss for the DRL-PE-GAT (50) and AM (50) are shown in Fig. 6(a) and Fig. 6(b) respectively, in order to compare the performance of two algorithms in solving small-size instances of the TD-EV-PDPTW model.

In Fig. 6(a) and Fig. 6(b), the DRL-PE-GAT (50) can converge within 20 epochs in the training curves of both distance and spoilage loss, and the AM (50) take more than 20 epochs to converge. Besides, the DRL-PE-GAT (50) can obtain smaller values in both distance and spoilage loss than AM (50), which indicates that DRL-PE-GAT (50) outperforms the AM (50) in solving the small-size instances of the TD-EV-PDPTW model.

To validate the effectiveness of the proposed algorithm in solving large-size instances of the TD-EV-PDPTW model, Fig. 7(a) and Fig. 7(b) record the training curves of both distance and spoilage loss for the DRL-PE-GAT (100) and AM (100) respectively.

As shown in Fig. 7 (a) and Fig. 7(b), the DRL-PE-GAT (100) converges faster than the AM (100) in the training curves of both distance and spoilage loss, which indicates that the DRL-PE-GAT (100) has better convergence than AM (100). Moreover, the DRL-PE-GAT (100) can obtain smaller and stabler values of both distance and spoilage loss after convergence.

Next, the DRL-PE-GAT algorithm is compared with other three baseline algorithms on the testing benchmark instances. For the DRL-PE-GAT algorithm and AM, the greedy decoding strategy (Kool et al. 2019) is used. Table 2 records the indicators including the average values of objective function and computing time in seconds, denoted as “Avg” and “Time” respectively.

According to Table 2, when solving the instance “N10-R3-E3”, ALNS obtains the best average value of objective function among four algorithms, and there is little difference of computation time between four algorithms. However, while ALNS and VNS are effective for small and medium-sized instances, their reliance on iterative neighborhood search and lack of parallelization make them less suitable for the large-scale TD-EV-PDPTW model. Therefore, with the increase of problem scale, the search spaces of heuristic algorithms (i.e., ALNS and VNS) expand sharply, resulting dramatic decreases of algorithms' performances. In contrast, the DRL-PE-GAT algorithm obtains the best average values of objective function on 23 out of 24 instances, and consumes the least computation time on 21 out of 24 instances, indicating that the DRL-PE-GAT algorithm is more effective and efficient in solving the TD-EV-PDPTW model. For the DRL-PE-GAT algorithm, the closer the size of

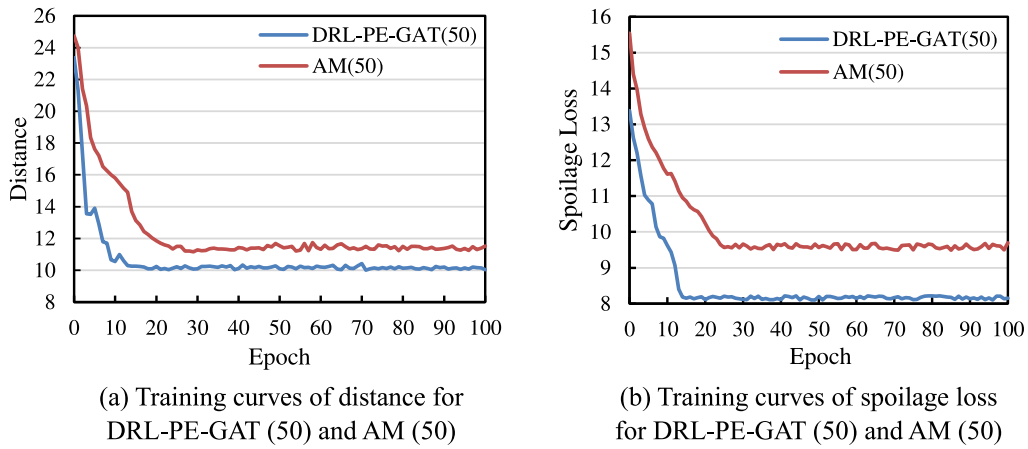


Fig. 6. Training curves for DRL-PE-GAT (50) and AM (50).

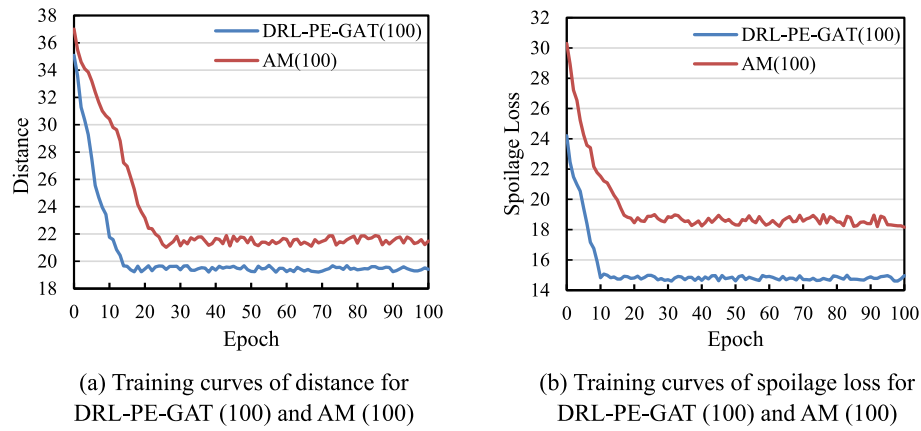


Fig. 7. Training curves for DRL-PE-GAT (100) and AM (100).

Table 2
Comparison results of four algorithms.

Instances	DRL-PE-GAT (50)		DRL-PE-GAT (100)		AM (50)		AM (100)		ALNS		VNS	
	Avg	Time	Avg	Time	Avg	Time	Avg	Time	Avg	Time	Avg	Time
N10-R3-E3	8.64	0.75	8.67	0.75	8.88	1.11	8.87	1.13	8.55	2.45	8.59	1.88
N30-R5-E5	14.91	1.90	15.21	2.03	15.76	2.00	15.74	1.91	15.06	47.57	15.92	60.98
N50-R7-E6	18.87	4.64	19.31	4.41	19.80	6.51	19.06	6.65	18.95	353.07	19.28	411.29
N70-R9-E8	21.20	5.38	21.32	5.43	22.21	7.87	22.32	8.34	22.17	1799.86	23.37	1943.63
N90-R11-E10	28.32	6.17	27.96	6.29	29.27	6.34	29.05	6.30	30.69	3047.61	32.11	>3600
N90-R15-E9	26.21	6.73	25.69	6.88	28.86	6.97	28.29	6.56	28.06	3502.52	30.23	>3600
N110-R13-E12	33.57	6.93	31.27	6.90	36.43	7.02	34.63	7.14	38.17	>3600	40.22	>3600
N110-R17-E11	31.19	7.17	29.85	8.26	34.60	7.55	33.18	7.90	36.45	>3600	38.83	>3600
N130-R15-E14	35.14	7.96	32.52	7.69	39.24	7.68	37.19	8.04	41.47	>3600	44.92	>3600
N130-R21-E12	33.74	8.09	31.24	8.60	36.56	8.80	35.95	8.77	39.36	>3600	41.52	>3600
N150-R17-E15	36.78	8.16	33.07	8.40	39.92	8.39	37.42	8.72	44.27	>3600	49.92	>3600
N150-R23-E13	35.44	8.46	31.99	8.57	37.06	8.90	35.82	9.19	39.81	>3600	42.48	>3600
N170-R19-E18	43.39	10.52	39.78	10.58	46.53	11.14	44.42	11.23	50.92	>3600	55.90	>3600
N170-R25-E15	40.96	10.57	37.50	10.64	43.18	11.95	41.92	11.00	48.46	>3600	51.51	>3600
N200-R21-E21	50.41	12.51	46.65	12.17	54.67	12.94	52.65	12.04	58.81	>3600	65.41	>3600
N200-R25-E20	49.63	12.79	46.37	12.94	55.86	12.74	52.51	12.07	55.04	>3600	61.18	>3600
N230-R25-E26	58.92	13.68	55.36	13.28	62.49	13.98	60.13	13.64	66.35	>3600	70.08	>3600
N230-R31-E23	54.73	13.19	51.42	13.30	59.94	14.23	57.67	14.04	64.32	>3600	68.98	>3600
N250-R27-E28	66.33	14.66	60.55	14.49	70.14	14.60	68.32	14.50	74.29	>3600	78.03	>3600
N250-R33-E25	63.01	14.36	55.14	14.29	68.84	14.58	65.01	14.48	67.16	>3600	72.84	>3600
N270-R29-E29	70.63	14.70	65.65	14.25	79.78	14.88	73.71	14.86	82.51	>3600	87.75	>3600
N270-R35-E27	67.21	14.55	61.38	14.05	73.32	14.96	69.01	14.26	79.31	>3600	82.57	>3600
N300-R31-E30	82.69	16.24	74.88	15.88	90.41	16.69	84.67	16.38	96.31	>3600	105.70	>3600
N300-R37-E26	77.86	16.01	70.27	15.73	86.90	16.34	81.49	16.75	90.13	>3600	97.32	>3600

Note: the best results on each instance are marked in bold.

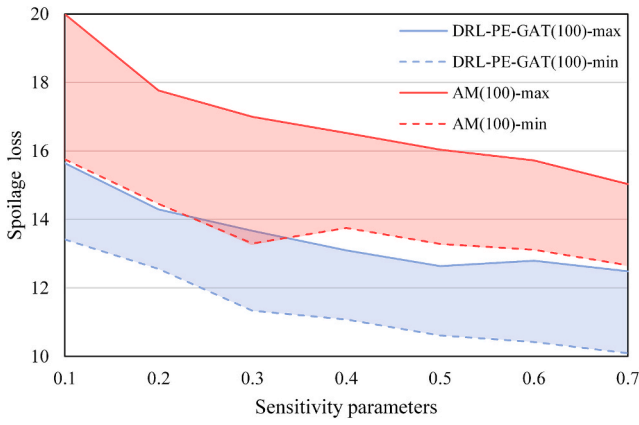


Fig. 8. Product spoilage loss under different sensitivity parameters.

Table 3
Numerical results under different product's lifespan.

β	DRL-PE-GAT (50)		DRL-PE-GAT (100)	
	Obj	Num	Obj	Num
30 %	76.43	68	70.79	63
40 %	64.87	50	59.45	44
50 %	59.98	41	53.20	32
60 %	50.41	35	46.65	26
70 %	47.04	32	44.23	24
80 %	46.07	30	39.92	19

testing benchmark instances is to that of training benchmark instances, the better the solution quality is.

5.3. Sensitivity analyses of the TD-EV-PDPTW model

Since the customers' sensitivity to different product spoilage differs, this study conducts the sensitivity analysis on the instance "N150-R17-E15" to simulate the real-world situations and analyze the impact of the sensitivity parameter ξ on the proposed model. For each sensitivity parameter, the DRL-PE-GAT (100) and AM (100) are both run five times. Fig. 8 records the maximum and minimum spoilage losses using DRL-PE-GAT (100) and AM (100) under different sensitivity parameters.

According to Fig. 8, product spoilage loss tends to decrease as the sensitivity parameter increases. Moreover, compared with AM (100), the DRL-PE-GAT (100) can provide smaller spoilage loss, and a narrower gap between the maximum and minimum spoilage losses, indicating that DRL-PE-GAT (100) obtains better and stabler solutions for the TD-EV-PDPTW model.

As for decision-makers, sensitivity parameter is a key factor that affects EV's routing plan and ultimately influences the loss of product spoilage, which should be considered during cold supply chains transportation.

In reality, different kinds of products have different lengths of lifespan. This study conducts the sensitivity analysis of product's lifespan β on the instance "N200-R21-E21" to analyze its impact on the proposed model. The value of β is set to 30 % to 80 % of EV's maximum travel time horizon. Table 3 records the numerical results of the objective function value and number of unsalable products, which is denoted as "Obj" and "Num" respectively.

According to Table 3, when β is small, the value of objective function is high, and numerous products become unsalable due to short lifespan. With the increase of length of lifespan, more products become salable, and EV's routing plan becomes more flexible, resulting in smaller objective function value. For decision-makers, when transporting products with short length of lifespan, various methods need to be employed to enhance efficient transport (e.g., using more EVs at the

same time, avoiding traffic congestion periods).

6. Conclusion

Efficient routing plan plays an important role in cold supply chains. This study focuses on the time-dependent travel time, and proposes the TD-EV-PDPTW model. To obtain the time-dependent travel time through the predicted travel speed, this study employs SVM to train the travel speed predictor using the real-world traffic data. To provide a precise evaluation of product spoilage loss, the traditional linear loss function for product spoilage is extended by taking account into the customers' sensitivity to product spoilage. Considering the characteristic of the TD-EV-PDPTW model, the DRL-PE-GAT algorithm incorporating the paired embedding and GAT is proposed to solve the model effectively. The paired embedding and GAT are utilized to capture the intrinsic relationship between pickup and delivery customer nodes and obtain the spatial dependencies between nodes, respectively.

To validate the performance of the DRL-PE-GAT algorithm, this study conducts several comparison experiments with three baseline algorithms. The numerical results show that the DRL-PE-GAT algorithm has better convergence and provides higher-quality solutions with shorter time. The sensitivity analysis discovers that sensitivity parameter and product's lifespan have great impact on the spoilage loss and optimal routing plans.

In the future, the TD-EV-PDPTW model can be extended in the following aspects. Firstly, different kinds of products delivered by the same EV can be considered in the model to improve the transportation efficiency. Secondly, other prediction models such as the long-short term memory and recurrent neural network can be used to capture the intrinsic relationship of traffic data with higher precision. Finally, the heuristic algorithm is a promising approach to be integrated with DRL-PE-GAT algorithm, so as to improve the algorithmic solution further.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors state that this research complies with ethical standards. This research does not involve either human participants or animals.

Data availability

I have declared the source of the experimental data in the experimental section of the paper.

References

- Archetti, C., Speranza, M. G., Boccia, M., Sforza, A., & Sterle, C. (2020). A branch-and-cut algorithm for the inventory routing problem with pickups and deliveries. *Eur. J. Oper. Res.*, 282(3), 886–895.
- Baldacci, R., Bartolini, E., & Mingozzi, A. (2011). An exact algorithm for the pickup and delivery problem with time windows. *Oper. Res.*, 59(2), 414–426.
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Mach. Learn.*, 20, 273–297.
- Delamer, J. A., Cupido, K., & Givigi, S. (2024). Graph neural network to solve the dynamic emergency vehicles routing problems. In *Proceedings of IEEE International Systems Conference*, April 15–18, Montreal, Canada. pp. 2472–9647.
- Deng, F., Qin, H., Li, J., & Cheng, C. (2022). The pickup and delivery problem with time windows and incompatibility constraints in cold chain transportation. *Transp. Sci.*, 57(2), 444–462.
- Dumas, Y., Desrosiers, J., & Soumis, F. (1991). The pickup and delivery problem with time windows. *Eur. J. Oper. Res.*, 54(1), 7–22.
- Figliozzi, M. A. (2011). The impacts of congestion on time-definitive urban freight distribution networks CO2 emission levels: Results from a case study in Portland, Oregon. *Transp. Res. Part C Emerging Technol.*, 19(5), 766–778.

- Gmira, M., Gendreau, M., Lodi, A., & Potvin, J. Y. (2021). Tabu search for the time-dependent vehicle routing problem with time windows on a road network. *Eur. J. Oper. Res.*, 288(1), 129–140.
- Guo, F., Wei, Q., Wang, M., Guo, Z. X., & Wallace, S. W. (2023). Deep attention models with dimension-reduction and gate mechanisms for solving practical time-dependent vehicle routing problems. *Transportation Research Part E: Logistics and Transportation Review*, 173, Article 103095.
- Harbaoui Dridi, I., Ben Alaïa, E., Borne, P., & Bouchriha, H. (2020). Optimisation of the multi-depots pick-up and delivery problems with time windows and multi-vehicles using PSO algorithm. *Int. J. Prod. Res.*, 58(14), 4201–4214.
- Hashemi-Amiri, O., Ghorbani, F., & Ji, R. (2023). Integrated supplier selection, scheduling, and routing problem for perishable product supply chain: A distributionally robust approach. *Comput. Ind. Eng.*, 175, Article 108845.
- Kool, W., Van Hoof, H., & Welling, M. (2019). Attention, learn to solve routing problems. In *Proceedings of the 7th International Conference on Learning Representations*, May 6–9, New Orleans, USA, pp. 1–25.
- Lei, K., Guo, P., Wang, Y., Wu, X., & Zhao, W. C. (2022). Solve routing problems with a residual edge-graph attention neural network. *Neurocomputing*, 508, 79–98.
- Leng, L. L., Zhang, J. L., Zhang, C. M., Zhao, Y. W., Wang, W. L., & Li, G. F. (2020). Decomposition-based hyperheuristic approaches for the bi-objective cold chain considering environmental effects. *Comput. Oper. Res.*, 123, Article 105043.
- Li, J. W., Xin, L., Cao, Z. G., Lim, A., Song, W., & Zhang, J. (2021). Heterogeneous attentions for solving pickup and delivery problem via deep reinforcement learning. *IEEE Trans. Intell. Transp. Syst.*, 23(3), 2306–2315.
- Li, W. L., Li, K. P., Kumar, P. R., & Tian, Q. N. (2021). Simultaneous product and service delivery vehicle routing problem with time windows and order release dates. *App. Math. Model.*, 89, 669–687.
- Lin, B., Ghaddar, B., & Nathwani, J. (2021). Deep reinforcement learning for the electric vehicle routing problem with time windows. *IEEE Trans. Intell. Transp. Syst.*, 23(8), 11528–11538.
- Liu, W. J., Zhou, Y. T., Liu, W., Qiu, J., Xie, N. M., Chang, X. Y., & Chen, J. (2021). A hybrid ACS-VTM algorithm for the vehicle routing problem with simultaneous delivery & pickup and real-time traffic condition. *Comput. Ind. Eng.*, 162, Article 107747.
- Luo, J., & Li, C. F. (2025). An efficient encoder-decoder network for the capacitated vehicle routing problem. *Expert Syst. Appl.*, 278, Article 127311.
- Masood, Z., Jiangbin, Z., Ahmad, I., Dongdong, C., Shabbir, W., & Irfan, M. (2024). A novel continual reinforcement learning-based expert system for self-optimization of soft real-time systems. *Expert Syst. Appl.*, 238, Article 122309.
- Mladenović, N., & Hansen, P. (1997). Variable neighborhood search. *Comput. Oper. Res.*, 24(11), 1097–1100.
- Nazari, M., Oroojlooy, A., Snyder, L., & Takác, M. (2018). Reinforcement learning for solving the vehicle routing problem. In *Proceeding of Advances in Neural Information Processing Systems*, Montréal, Canada, December 2–8, pp. 9839–9849.
- Osvald, A., & Stirn, L. Z. (2008). A vehicle routing algorithm for the distribution of fresh vegetables and similar perishable food. *J. Food Eng.*, 85(2), 285–295.
- Qian, J. N., & Eglese, R. (2016). Fuel emissions optimization in vehicle routing problems with time-varying speeds. *Eur. J. Oper. Res.*, 248(3), 840–848.
- Rahbari, A., Nasiri, M. M., Werner, F., Musavi, M., & Jolai, F. (2019). The vehicle routing and scheduling problem with cross-docking for perishable products under uncertainty: Two robust bi-objective models. *App. Math. Model.*, 70, 605–625.
- Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transp. Sci.*, 40(4), 455–472.
- Shi, Y., Boudouh, T., Grunder, O., & Wang, D. Y. (2018). Modeling and solving simultaneous delivery and pick-up problem with stochastic travel and service times in home health care. *Expert Syst. Appl.*, 102, 218–233.
- Song, M. X., Li, J. Q., Han, Y. Q., Han, Y. Y., Liu, L. L., & Sun, Q. (2020). Metaheuristics for solving the vehicle routing problem with the time windows and energy consumption in cold chain logistics. *Appl. Soft Comput.*, 95, Article 106561.
- Stellingwerf, H. M., Groeneveld, L. H., Laporte, G., Kanellopoulos, A., Bloemhof, J. M., & Behdani, B. (2021). The quality-driven vehicle routing problem: Model and application to a case of cooperative logistics. *Int. J. Prod. Econ.*, 231, Article 107849.
- Sun, P., Veelenturf, L. P., Hewitt, M., & Van Woensel, T. (2018). The time-dependent pickup and delivery problem with time windows. *Transp. Res. B Methodol.*, 116, 1–24.
- Veenstra, M., Cherkies, M., Desaulniers, G., & Laporte, G. (2017). The pickup and delivery problem with time windows and handling operations. *Comput. Oper. Res.*, 77, 127–140.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2018). Graph attention networks. In *Proceedings of the 6th International Conference on Learning Representations*, April 30–May 3, Vancouver, Canada, pp. 1–12.
- Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. In *Proceeding of Advances in Neural Information Processing Systems*, Montréal, Canada, December 7–12, pp. 2692–2700.
- Wang, Q., Li, H., Wang, D. J., Cheng, T. C. E., & Yin, Y. Q. (2023). Bi-objective perishable product delivery routing problem with stochastic demand. *Comput. Ind. Eng.*, 175, Article 108837.
- Wang, X. Y., Choi, T. M., Li, Z. Y., & Shao, S. (2019). An effective local search algorithm for the multidepot cumulative capacitated vehicle routing problem. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 50(12), 4948–4958.
- Wang, Y., Peng, S. G., Guan, X. Y., Fan, J. X., Wang, Z., Liu, Y., & Wang, H. Z. (2021). Collaborative logistics pickup and delivery problem with eco-packages based on time-space network. *Expert Syst. Appl.*, 170, Article 114561.
- Wang, Y., Wang, Z., Hu, X. P., Xue, G. Q., & Guan, X. Y. (2022). Truck–drone hybrid routing problem with time-dependent road travel time. *Transp. Res. Part C Emerging Technol.*, 144, Article 103901.
- Zhang, S., Xia, J. M., Chen, Q. J., & Zhang, W. Y. (2024). A two-phase adaptive large neighborhood search algorithm for the electric location routing problem with range anxiety. *Appl. Soft Comput.*, 167, Article 112323.
- Zheng, J., Wang, L., Wang, L., Wang, S. Y., Chen, J. F., & Wang, X. (2022). Solving stochastic online food delivery problem via iterated greedy algorithm with decomposition-based strategy. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 53(2), 957–969.
- Zong, Z. F., Zheng, M., Li, Y., & Jin, D. P. (2022). MAPDP: Cooperative multi-agent reinforcement learning to solve pickup and delivery problems. In *Proceedings of the 36th AAAI Conference on Artificial Intelligence*, February 22–March 1, Pennsylvania, USA, pp. 9980–9988.